

CSE 6220 High Performance Computing
Programming Assignment-3 Report
Submitted April 14, 2025

Name: Honglin Liu
Name: Xuzheng Tian

gID: 903651409
gID: 903618682

1 Implementation of the 2D SUMMA SpGeMM Algorithm

Our implementation of the Sparse General Matrix-Matrix Multiplication (SpGeMM) algorithm computes $C = A \times B$ for sparse matrices A ($m \times p$) and B ($p \times n$) in COO (Coordinate List Format) format, using a 2D $\sqrt{p} \times \sqrt{p}$ processor grid.

We adopt the Sparse [SUMMA](#) (Scalable Universal Matrix Multiplication) algorithm, optimized for non-zero elements, with key functions implemented in `SpGeMM.cpp`.

1.1 Algorithm implementation

The algorithm proceeds as follows:

1. Initialization:

The root processor distributes input matrices A and B in global COO format to the $\sqrt{p} \times \sqrt{p}$ processor grid, with each processor receiving entries for its block without coordinate adjustments.

The `SpGeMM_2d` function determines the local block dimensions for each processor.

For a processor at grid position `(row_rank, col_rank)`, we compute block sizes as `block_size_m = (m+size-1)/size`, `block_size_n`, and `block_size_p`, where `size` is the grid dimension ($\sqrt{p}$).

A local matrix `C_block` is created as a sparse map, with elements implicitly set to 0 (non-existent entries are treated as 0 during computation).

Actual block sizes `(current_size_m, current_size_n, current_size_p)` adjust for boundary conditions.

2. COO to CSR Conversion:

To optimize non-zero element access, `COO2CSR` converts input matrices A and B from COO to CSR (Compressed Sparse Row) format.

For a matrix with `length` non-zeros, `COO2CSR` allocates `row_ptr` (`size+1`), `idx`, and `values` (`size length`).

It counts non-zeros per row and computes a prefix sum for `row_ptr`, enabling efficient row-wise traversal.

3. SUMMA Computation:

The SUMMA function executes the core multiplication.

For each iteration k (0 to `col_size`):

- **Broadcast A :**

The k -th column block of A (CSR format) is broadcast within the row communicator (`row_comm`) using `MPI_Bcast`.

The block includes `row_ptr`, `idx`, and `values`, with size determined by non-zero count (`nnz_A`).

- **Broadcast B :**

The k -th row block of B is broadcast within the column communicator (`col_comm`).

- **Local Computation:**

Each processor computes a contribution to C 's local block (`C_block`, a sparse map of coordinates to values).

For each non-zero $A[i, A_col]$ and $B[A_col, B_col]$, we update `C_block[{i, B_col}] = plus(C_block[{i, B_col}], times(A[i, A_col], B[A_col, B_col]))`, where `plus` and `times` define the semiring.

`C_block` is initialized as an empty `std::map` in `SpGeMM_2d`, storing only non-zero entries.

4. Output Conversion:

The `MTX2COO` function converts `C_block` to COO format.

It iterates over non-zero entries in the sparse map, using their global coordinates directly to form the output matrix C .

For APSP, `SpGeMM_2d` is used with the min-plus semiring, treating undeclared values in `C_block` as infinity and setting diagonal entries to 0 to compute shortest paths correctly.

The implementation ensures correctness for arbitrary $m \times p$ and $p \times n$ matrices while supporting flexible semiring operations, as tested with $A \times A^T$.

2 Implementation of the APSP Algorithm

The APSP (All-Pairs Shortest Path) algorithm computes shortest path distances between all vertex pairs in a weighted graph with n nodes, represented in COO format.

Inspired by the [Floyd-Warshall](#) algorithm, our implementation in `apsp.cpp` uses matrix multiplication with a *min-plus* semiring operator, executed via `SpGeMM_2d`.

The graph described in the problem is modeled as an $n \times n$ distance matrix L , where $L[i, j]$ is the edge weight from node i to j (0 denotes infinity).

2.1 Implementation of min-plus semiring

The *min-plus* semiring redefines operations:

- `plus`:
 $\min(a, b)$, selecting the shorter path. If $a = 0$ (infinity), returns b ; otherwise, returns the minimum of a and b .
- `times`:
 $a + b$, summing edge weights directly, assuming valid weights (zeros as 0).

This allows $L^k[i, j]$ to represent the shortest path from i to j using at most k intermediate nodes. Diagonal entries are set to 0 after each iteration to ensure correctness.

2.2 Algorithm implementation

The algorithm proceeds as follows:

1. Initialization:

In `apsp`, we set L to the input graph and compute local block ranges for each processor (`row_rank`, `col_rank`) in the $\sqrt{p} \times \sqrt{p}$ grid.

The block size is `block_size_n = (n+row_size-1)/row_size`, with row range `[row_start, row_end)` and column range `[col_start, col_end)`.

2. Iterative Matrix Multiplication:

For iterations where `max_iter` doubles from 1 to n :

- Copy L to L_{tmp} .
- Call `SpGeMM_2d` to compute $L = L_{tmp} \times L_{tmp}$ under the min-plus semiring, updating $L[i, j] = \min(L[i, j], \min_k(L[i, k] + L[k, j]))$.
- Set diagonal entries $L[i, i]$ to 0 to correct potential errors from summation or minimization.

3. Local Processing:

Each processor retains L s entries in global coordinates, with no additional filtering required beyond diagonal corrections performed in step 2.

4. **Synchronization:**

`MPI_Barrier` ensures processors align after each iteration.

5. **Output:**

Global coordinates are restored by adding `row_start` and `col_start` offsets, yielding the result in COO format.

This *Floyd-Warshall*-inspired approach leverages `SpGeMM_2ds` parallel efficiency to compute shortest paths. The *min-plus* semiring ensures correctness, while local processing reduces communication overhead.

3 Analysis of Algorithms' Performances

We analyze the theoretical computational complexity and communication costs; with PACE clusters, we also record the actual performances of our SpGeMM (SpGeMM_2d) and APSP (apssp) implementations on a $\sqrt{p} \times \sqrt{p}$ processor grid with $P = 1^2, 2^2, 3^2, 4^2, 5^2, 6^2$.

3.1 Complexity and Communication

SpGeMM:

Comput. cost:

$O((\text{nnz}(A) \cdot \text{nnz}(B))/p + \sqrt{p})$ for one matrix multiplication.

Comm. cost:

$\sqrt{p}$ MPI_Bcast calls cost $O(\tau \cdot \sqrt{p} + \mu \cdot (\text{nnz}(A) + \text{nnz}(B))/\sqrt{p})$ per processor

APSP:

Comput.cost:

$O((\text{nnz}(G)^2 \cdot \log n)/p + \sqrt{p} \cdot \log n)$ for $\text{nnz}(G)$ edges, due to $\lceil \log n \rceil$ SpGeMM_2d calls.

Comm. cost:

$\log n \times \text{SpGeMMs cost} + \text{MPI_Barrier}$, totaling $O(\tau \cdot \sqrt{p} \cdot \log n + \mu \cdot \text{nnz}(G) \cdot \log n / \sqrt{p})$ per processor.

*: nnz *abbr.* for “number of non-zero elements”; $G := \text{Graph}$; τ is latency; μ is bandwidth.

Comparison:

SpGeMM is lighter (single pass) but scales similarly to APSP per iteration. APSPs $\log n$ iterations inflate computation and communication, reducing scalability for large n .

3.2 Performance visualization

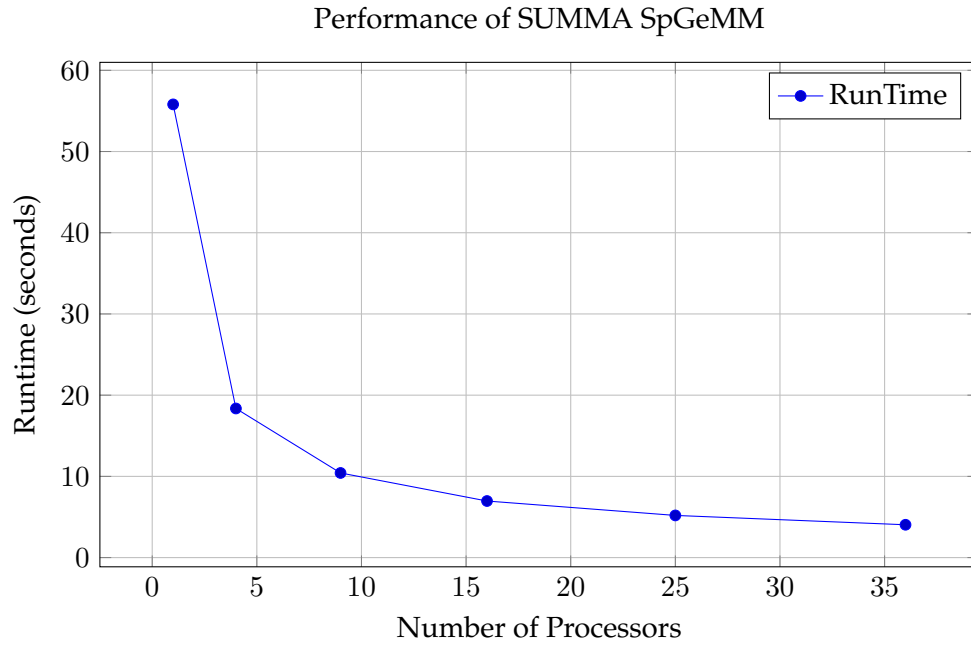


Figure 1: SpGeMM: 55.80s to 4.04s ($p = 36$, speedup $13.80\times$).

SpGeMM Scales from 55.80s ($p = 1$) to 4.04s ($p = 36$), meeting < 10 s target. Speedup ($13.80\times$) slows beyond $p = 16$ due to communication ($O(\alpha \cdot \sqrt{p})$).

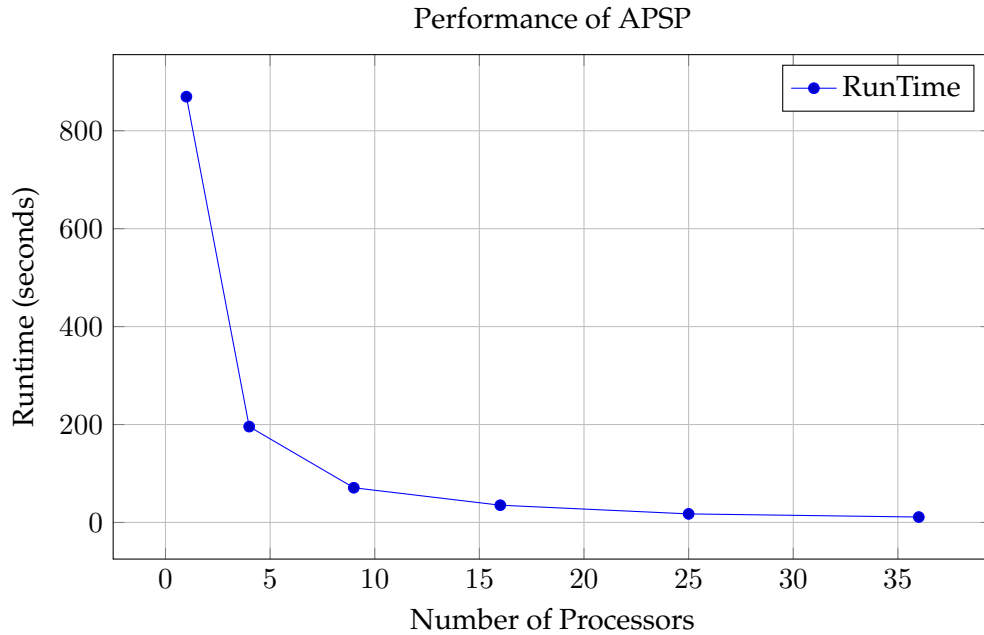


Figure 2: APSP: 869.63s to 11.01s ($p = 36$, speedup $79.02\times$).

APSP Drops from 869.63s to 11.01s, exceeding < 30 s target. Higher speedup ($79.02\times$) reflects more enormous sequential workload, but $\log n$ iterations limit scalability.

APSPs iterative cost yields higher runtimes but better speedup. Both face load imbalance and communication bottlenecks at high p .